

CSE 3300 Project 2 Design Report

Description: There are two main parts to my program – the client and the server. The client is very simple: it creates a socket connection, connects to port 26382, and sends query requests to the server on port 26382. I have the query part nested in a while loop, so the user can make as many queries as they want and quits by submitting exit, which breaks out of the loop and closes the connection.

The server is larger and has multiple functions to handle all the functionality. It is hosted on port 26382, so when I run it I am creating a server on port 26382. It first opens the wordlist.txt file and saves it into a set for queries. It then creates the server socket and allows for up to five queued connections. When client connections are established, it creates a new thread to handle each client.

To handle a client, the server parses the message and queries for matching strings. I converted the ? to a ., which is the Python regex library wildcard character. I then query for words that are the same size and have the wildcard character in wordlist using regex. It then sends it to the client. If the query is bad or there are no results, it sends those messages to the client.

The application protocol I based my code off of was HTTP. My code uses the same header format to send messages and the client sends a GET request. The server's responses are HTTP status codes: 200 OK if there was a successful query, 204 No Content if the query returned no results, and 400 Bad Request if there was an issue with the client's query (like improper format). I also used TCP sockets like HTTP does for reliable connections.

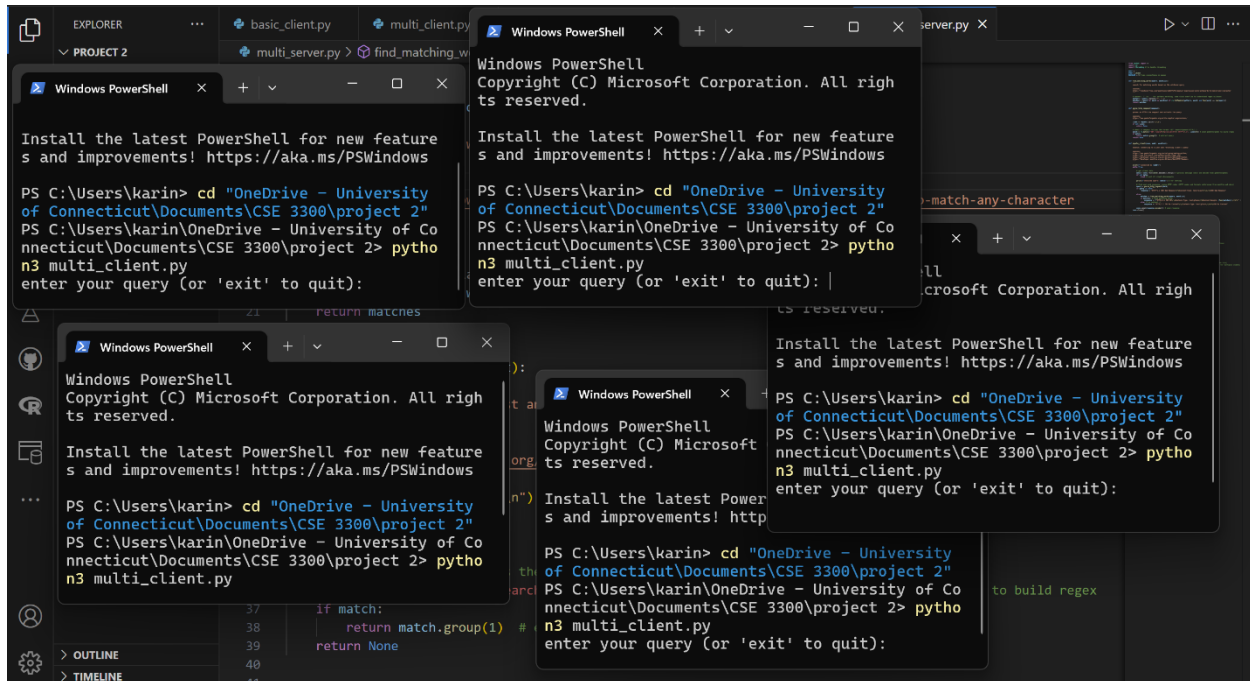
Tradeoffs: The biggest tradeoff was how I queried and loaded words. Every time I start the server, it loads the entire wordlist and each query checks the entire wordlist against the query. A more optimized search algorithm could speed up the queries and ensure my server was using less resources to find the words. I considered researching a better algorithm than brute force searching, but I liked the simplicity and readability of my code right now. The wordlist is not big enough for the code to noticeably slow down queries, but what I have isn't scalable if the wordlist gets bigger.

Extensions: There are a lot of things I wanted to add but these are the biggest features I think would make my program better:

- The ability to post or remove words to the wordlist as a client. It would be nice if clients could update the list of words in case it is missing a word or there is a typo. Currently the only way to update it is by editing the text file. I would do this by expanding the client code and allowing them to make post and remove requests to the server's wordlist instead of just gets.
- More specific error messages. Currently it will just throw a 400 Bad Request if it's a badly formatted request, but that could mean the header was wrong or there was a space or numerical character, the client doesn't know what they did wrong. I would go into my HTTP parsing function and check for more specific errors and send specific messages.

Test cases:

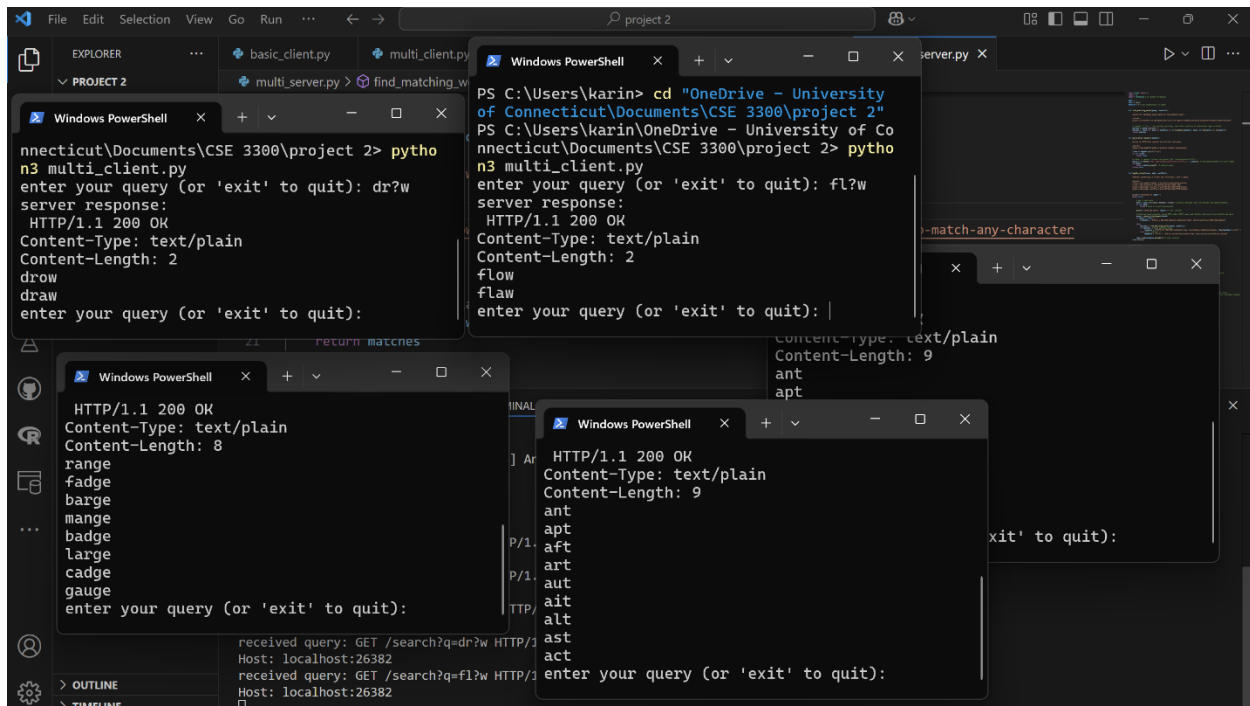
First test: making sure the server can connect to multiple clients. I ran the server and then created five connections and ensured all of them could connect to my client. The “enter your query” means they were all able to successfully connect.



```
PS C:\Users\karin> cd "OneDrive - University of Connecticut\Documents\CSE 3300\project 2"
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit):
```

```
def find_matching_word(word):
    for i in range(1, len(word)):
        if word[i] == ' ':
            return word[i+1:]
    return None
```

Second test: making sure all of them were able to successfully query. All of the clients I connected to the server did successfully make queries and the server outputted all the queries as well.



```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): dr?w
server response:
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
draw
draw
enter your query (or 'exit' to quit):
```

```
def find_matching_word(word):
    for i in range(1, len(word)):
        if word[i] == ' ':
            return word[i+1:]
    return None
```

Third test: all HTTP messages work. If I submit a valid query, it gives me a 200 OK and the list of words. If I submit a valid query but with no matches, it returns a 204 No Content. If I submit an invalid query, it throws a 400 Bad Request.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): fl?w
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
flow
flaw
enter your query (or 'exit' to quit): xxtrx
server response:
  HTTP/1.1 204 No Content
Content-Type: text/plain

204 No Content
enter your query (or 'exit' to quit): 1234
server response:
  HTTP/1.1 400 Bad Request
Content-Type: text/plain

400 Bad Request
```

Fourth test: can handle multiple wildcards. I put 2 and 5 wildcards and it was able to return words that satisfied the query.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): ?ai?e
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 7
faire
naive
zaire
baize
maine
waive
raise
enter your query (or 'exit' to quit): fath??
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
fathom
father
enter your query (or 'exit' to quit): tw??t
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 3
twist
tweet
twixt
enter your query (or 'exit' to quit): sa??t???
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 12
sanitary
sagittal
sanction
salutary
saboteur
sanctity
sanctify
saratoga
sanatory
sanation
sabotage
sanative
enter your query (or 'exit' to quit): |
```

Fifth test: case sensitivity. I did not implement handling cases so my server does not query properly when clients submit queries with uppercase characters.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): afrA?d
server response:
  HTTP/1.1 204 No Content
  Content-Type: text/plain

204 No Content
enter your query (or 'exit' to quit): afra?d
server response:
  HTTP/1.1 200 OK
  Content-Type: text/plain
  Content-Length: 1
afraid
enter your query (or 'exit' to quit): |
```

Sixth test: only wildcard characters. Submitting only wildcard characters does not break my server.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): ?????????????????????????????
server response:
  HTTP/1.1 204 No Content
  Content-Type: text/plain

204 No Content
enter your query (or 'exit' to quit): ?
server response:
  HTTP/1.1 200 OK
  Content-Type: text/plain
  Content-Length: 21
y
i
e
d
m
v
l
t
g
q
u
o
s
c
a
p
n
r
b
x
z
enter your query (or 'exit' to quit): |
```

Seventh test: no wildcard characters. My server returns the exact word (if it is in the wordlist) when no wildcard is in the query.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 multi_client.py
enter your query (or 'exit' to quit): precise
server response:
  HTTP/1.1 200 OK
  Content-Type: text/plain
  Content-Length: 1
precise
enter your query (or 'exit' to quit): |
```

Extra Credit Test Cases:

This screenshot demonstrates that it will query for any size string that contains the substring a?t in it. I could not fit all the output, but the Content-Length shows that there were 4260 words.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 extra_credit_client.py
enter your query (or 'exit' to quit): a?t
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 4260
castles
santalum
antiphlogistic
hindquarters
protractile
particula
aeternum
altogether
exuberant
flippantly
flattering
actinomyxidial
broadcaster
mastoid
afterimage
postulant
hyperactivity
heartswelling
```

If I just submit a wildcard, it outputs all 69,903 words in the wordlist.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 extra_credit_client.py
enter your query (or 'exit' to quit): ?
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 69903
castles
deferred
dancing
brinness
deuteronomy
hardheaded
anaclinal
inhalation
babysitting
gamete
santalum
tip-off
trustee
canaliferous
vacatur
bluehead
traveling
```

In this, you can see that the content-length header accurately shows the number of words outputted, as well as also shows that it gets both words that contain “pr?perly” in the wordlist.

```
PS C:\Users\karin\OneDrive - University of Connecticut\Documents\CSE 3300\project 2> python3 extra_credit_client.py
enter your query (or 'exit' to quit): pr?perly
server response:
  HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
improperly
properly
enter your query (or 'exit' to quit): |
```